Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(Московский Политех)

Лабораторная работа № 1
По курсу: Технологии конструирования программного обеспечения

Выполнил: Стальмахов Иван Сергеевич
Группа: 241-3211
Проверил: _____

МОСКВА
2026

Задание на лабораторную работу

Разработка программ с использованием принципа единственной обязанности (SRP) и принципа открытости/закрытости (OCP).

Вариант 1. Система Факультатив. Преподаватель объявляет запись на Курс. Студент записывается на Курс, обучается и по окончании Преподаватель выставляет Оценку, которая сохраняется в Архиве. Студентов, Преподавателей и Курсов при обучении может быть несколько.

Требуется: UML-диаграмма классов и диаграмма последовательности; консольное приложение (C#) с применением SRP и OCP; тестирование функциональности созданных классов.

Реализация SRP/OCP в программе: каждый класс имеет одну обязанность (Teacher — объявление записи и оценивание, Course — ведение записей, Student — запись на курс, Enrollment — факт записи, Grade — оценка, InMemoryArchive — хранение); OCP — интерфейс `IArchive` (`save`, `findByStudent`) открыт для расширения новыми реализациями архива без изменения существующего кода.

Диаграмма классов

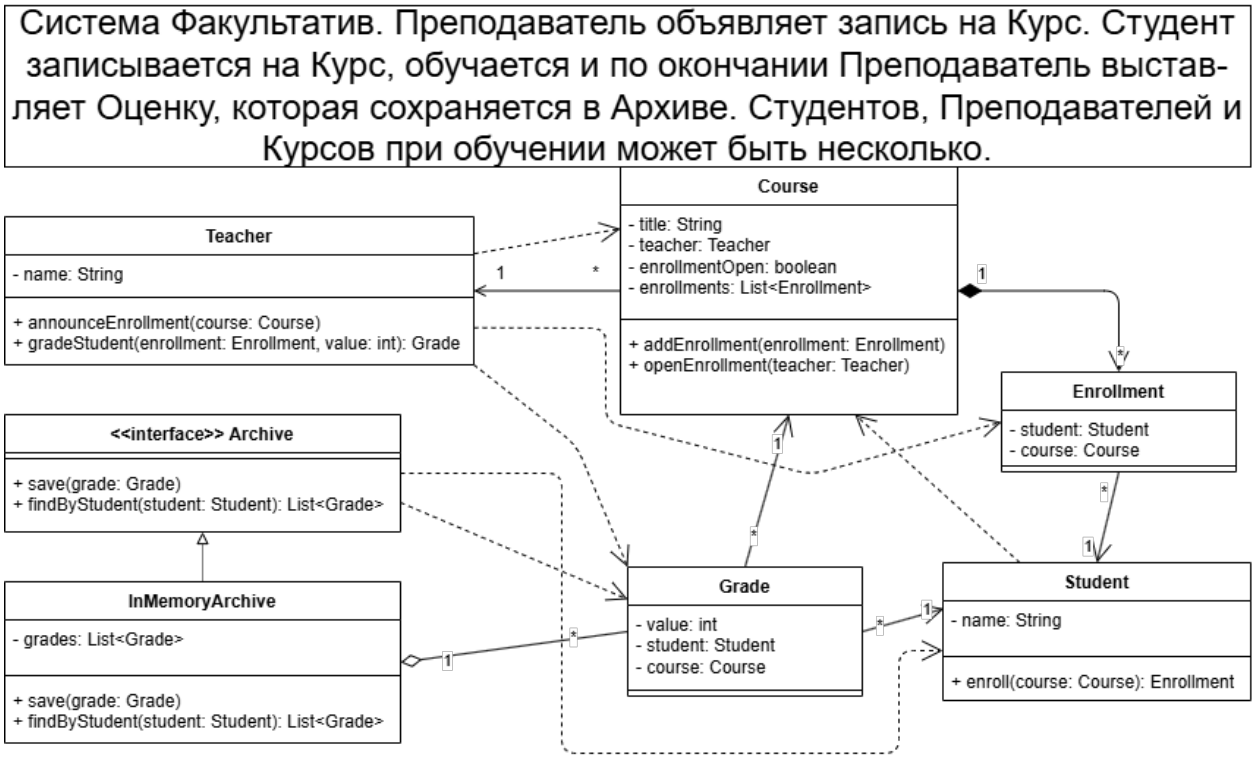
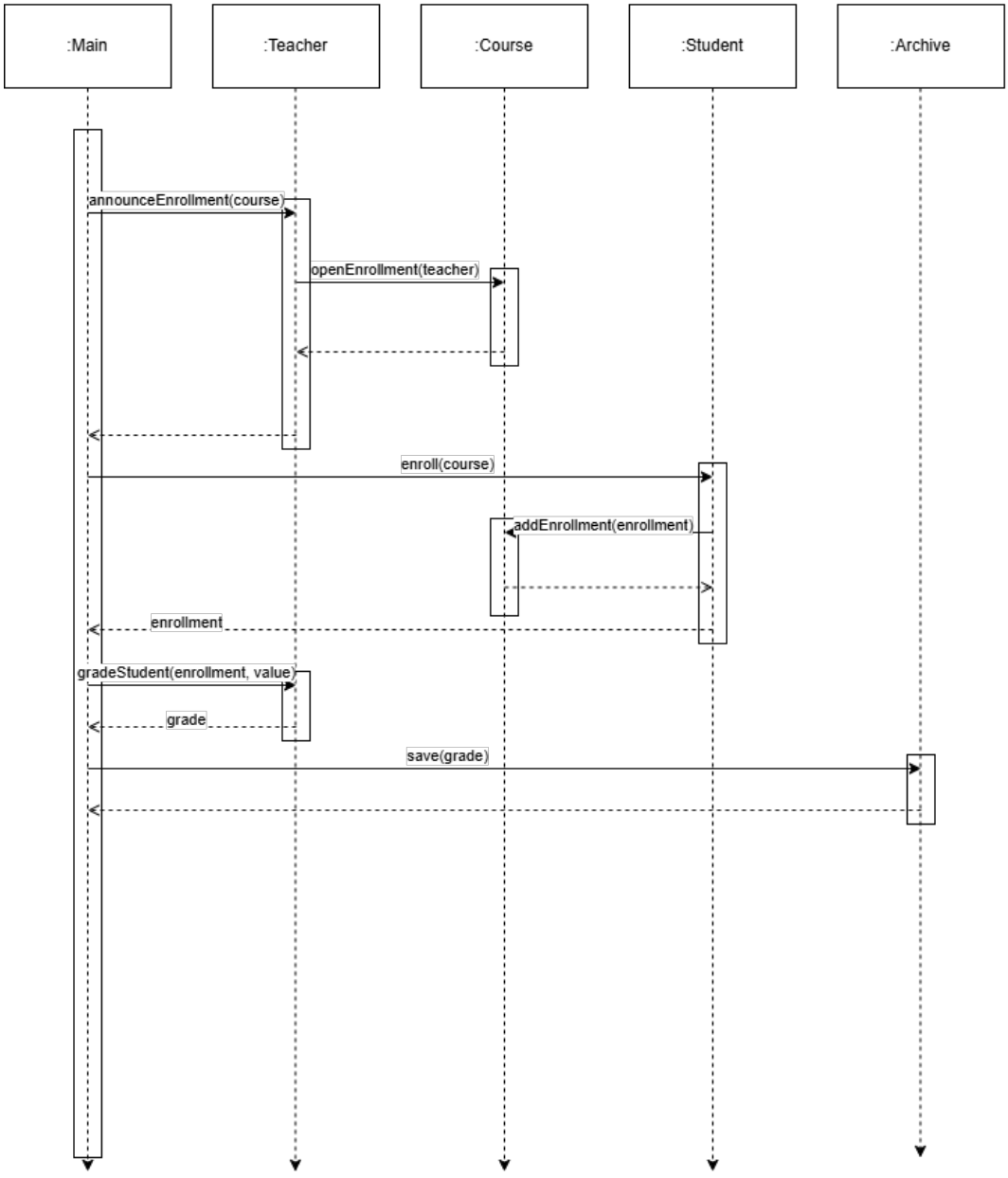


Диаграмма последовательности

Система Факультатив. Преподаватель объявляет запись на Курс. Студент записывается на Курс, обучается и по окончании Преподаватель выставляет Оценку, которая сохраняется в Архиве. Студентов, Преподавателей и Курсов при обучении может быть несколько.



Исходный код программы

```
namespace Lab1;
public class Teacher(string _name)
```

```
{

    public string Name{get;} = _name;


    public void announceEnrollment(Course course)
    {
        course.openEnrollment(this);
    }
    public Grade gradeStudent(Enrollment _enrollment, int _value)
    {
        return new Grade(_value, _enrollment.Student, _enrollment.
   Course);
    }

    public override string ToString() => Name;

}
```

```
namespace Lab1;
public class Student(string _name)

{
public string Name{get;} = _name;

public Enrollment enroll(Course _course)
    {
        return new Enrollment(this,_course);
    }
    public override string ToString() => Name;
}
```

```
namespace Lab1;
public class Course(string _title, Teacher _teacher)
{
    public string Title {get;} = _title;
    public Teacher Teacher{get;} = _teacher;
    public bool EnrollmentOpen{get; private set;}
    public List<Enrollment> Enrollments {get;} =[];
```

```
    public void openEnrollment(Teacher _teacher)
    {// дописать валидацию учителя.
    EnrollmentOpen = true;

    }
    public void addEnrollment(Enrollment _enrollment)
    {
        Enrollments.Add(_enrollment);
    }
    public override string ToString() => $"Название курса: {Title},
   Преподаватель: {Teacher}, Открыт набор: {EnrollmentOpen}, Записи [{
   Enrollments.Count}]: {string.Join("; ", Enrollments)}";
}
```

```
namespace Lab1;
public class Enrollment(Student _student, Course _course)

{
public Student Student{get;} = _student;
public Course Course{get;} = _course;


    public override string ToString() => $"Запись: {Student} на курсе
   \"{Course.Title}\"";
}
```

```
namespace Lab1;
public class Grade(int _value, Student _student, Course _course)
{


public int Value{get;} = _value;
public Student Student{get;} = _student;
public Course Course{get;}= _course;


    public override string ToString() => $"Оценка: {Value}, Студент: {
   Student.Name}, Курс: {Course.Title}";
```

```
}
```

```
namespace Lab1;
public interface IArchive
{
    public void save(Grade _grade);
    public List<Grade> findByStudent(Student _student);

}
```

```
namespace Lab1;
public class InMemoryArchive : IArchive
{
    private List<Grade> grades = [];
    public void save(Grade _grade)
    {
        grades.Add(_grade);
    }
    public List<Grade> findByStudent(Student _student)
    {
        return grades.FindAll(g => g.Student == _student);
    }
    public override string ToString() =>
        grades.Count == 0
            ? "Архив пуст"
            : string.Join(Environment.NewLine, grades);
}
```

```
using Lab1;

IArchive archive = new InMemoryArchive();

Teacher teacher = new("Василий");
Teacher teacher2 = new("Джабраил");

Course course = new("Мобильная Разработка", teacher);
Course course2 = new("ИКТ", teacher2);

Student student = new("Иван");
Student student2 = new("Сандер");
```

```
Console.WriteLine("перед: " + course);
Console.WriteLine("       "+ course2);
Console.WriteLine();


teacher.announceEnrollment(course);
teacher2.announceEnrollment(course2);

Enrollment enrollment = student.enroll(course);
Enrollment enrollment2 = student2.enroll(course2);

course.addEnrollment(enrollment);
course2.addEnrollment(enrollment2);

Grade grade = teacher.gradeStudent(enrollment, 5);
Grade grade2 = teacher2.gradeStudent(enrollment2, 2);

archive.save(grade);
archive.save(grade2);


Console.WriteLine("после: " + course);
Console.WriteLine("       "+ course2);
Console.WriteLine();
Console.WriteLine(archive);
Console.WriteLine();
Console.WriteLine("Оценки Ивана: " + string.Join("; ", archive.
   findByStudent(student)));
```

Результат запуска (тестирование функционально-сти)

```
перед
: Название курса: Мобильная Разработка, Преподаватель: Василий, Открыт набор: False,
   Записи [0]:
      Название курса: ИКТ, Преподаватель: Джабраил, Открыт набор: False, Записи [0]:
   после
```

```
: Название курса: Мобильная Разработка, Преподаватель: Василий, Открыт набор: True,
    Записи [1]: Запись: Иван на курсе Мобильная" Разработка"
       Название курса: ИКТ, Преподаватель: Джабраил, Открыт набор: True, Записи [1]:
    Запись: Сандер на курсе ИКТ""Оценка

: 5, Студент: Иван, Курс: Мобильная РазработкаОценка
: 2, Студент: Сандер, Курс: ИКТОценки

 Ивана: Оценка: 5, Студент: Иван, Курс: Мобильная Разработка
```